

好的，这个PPT介绍了**哈希（Hashing）**技术，它是一种用于高效查找、插入和删除数据的抽象数据类型。核心思想是实现 $O(1)$ 的平均时间复杂度。

查找问题 (Searching)

查找问题是在一个记录集合 L 中，找到键 k_j 等于给定键值 K 的记录 (k_j, I_j) 。

- **成功查找 (Successful Search):** 找到键 $k_j = K$ 的记录。
 - **不成功查找 (Unsuccessful Search):** 没有找到键 $k_j = K$ 的记录。
 - **查找类型:**
 - **精确匹配查询 (Exact-match query):** 查找键值与给定键值完全匹配的记录。
 - **范围查询 (Range query):** 查找键值落在给定范围内的所有记录。
-

传统查找方法及其性能 (Traditional Searching Methods)

传统的查找方法主要依赖于**比较操作**，通常的查找时间复杂度为 $O(\log N)$ 。

1. 顺序和列表方法 (Sequential and List Methods)

- 适用于存储在 **RAM (内存)** 中的数据。
- **未排序数组的顺序查找 (Sequential Search on Unsorted Lists):**
 - 最坏情况时间复杂度为 $\Theta(n)$ 。
 - **平均成本** $T_n: T_n = np_n + \sum_{i=0}^{n-1} (i+1)p_i$ 。
 - p_i 是 K 位于位置 i 的概率， p_n 是 K 不在列表中的概率。
- **自组织列表 (Self-Organizing Lists):**
 - 不按键值排序，而是按**预期访问频率**排序。
 - 将最常请求的记录放在列表的最前面。
 - **启发式策略 (Heuristic Strategies):** 用于在不知道访问频率的情况下重新排序。
 - **计数 (Count):** 记录访问次数并按此排序。
 - **移到前部 (Move-to-front):** 找到的记录立即移到列表头部。
 - **转置 (Transpose):** 找到的记录与其前一个记录交换位置。

2. 排序数组查找 (Searching Sorted Arrays)

- **二分查找 (Binary Search):** 在不知道键值分布的情况下，是查找排序数组的最佳方法。
 - 基于**分治 (Divide and Conquer)** 原理。
 - **跳跃查找 (Jump Search):** 检查 j 的倍数位置 $L[j], L[2j], \dots$ 。当 $L[mj] < K < L[(m+1)j]$ 时，在 $(\dots)$ 范围内顺序查找 $j-1$ 个元素。
 - 总比较次数 $T(n, j) = \lfloor n/j \rfloor + j - 1$ 。
 - **“计算”二分查找 / 字典查找 (“Computed” Binary Search / Dictionary Search):** 如果已知键的预期分布，例如根据字母表位置跳跃。
-

哈希表 (Hash Tables) - 追求速度

哈希表是一种抽象数据类型，旨在实现 $O(1)$ 的查找 (Find) 和插入 (Insert) 操作。

- **核心理念:** 通过一个函数将键直接映射到数组索引，实现**直接访问 (Direct Access)**。
- **有限操作集:** 许多应用只需要插入 (Insert)、查找 (Find) 和删除 (Delete) 操作，而**不要求元素有序**。

1. 直接地址表 (Direct Address Tables)

- **适用前提:**
 - 键是集合 $U = \{0, 1, \dots, m - 1\}$ 中的整数，且 m **很小**。
 - 没有两个元素具有相同的键。
- **实现:** 直接将元素存储在 $Table[key]$ 。
- **操作:** $Insert(T, x)$, $Delete(T, x)$, $Find(T, k)$ 都是直接的数组访问，时间复杂度为 $O(1)$ 。
- **问题 (Issue):** 如果键的宇宙 U 很大，而实际使用的键集合 K 很小 (即 $|U| \gg |K|$)，则数组非常**稀疏 (sparse)**，浪费空间，甚至可能无法在内存中容纳。

2. 哈希方案 (Hashing Schemes)

为了解决直接地址表的稀疏问题，需要将大的键宇宙 U 映射到一个与实际存储元素数量 $|K|$ 大小相近的小表 M 上。

- **哈希函数 (Hash Function $h(K)$):** 用于计算键 K 对应的表索引 (哈希值)。
 - 输出值必须小于表的大小 M 。
 - 应该**尽可能均匀分布 (evenly distributed)**，以最小化冲突。
 - 哈希值应依赖于整个键及其位置。
- **冲突解决策略 (Collision Resolution Strategy):** 用于处理两个不同键哈希到同一索引的情况。

哈希函数的选择 (Choosing the Hash Function)

1. 针对数值键 (Hashing Integers)

- **模哈希函数 (Mod Hash Function):** $Hash(key) = key \pmod{TableSize}$ 。
 - **注意:** $TableSize$ 最好选择**素数 (prime number)**，以避免键值之间有系统性关联而导致所有键映射到同一索引的问题 (例如，如果 $TableSize = 11$ ，所有两位重复的数字 22, 33, ... 都映射到同一索引 0)。
- **中平方法 (Mid-square method):** 将键平方，取结果的中间 r 位作为哈希值。

2. 针对非数值键 (Nonnumerical Keys)

- 需要先将非数值键 (如字符串) **转换为自然数**，通常使用 **ASCII 字符编码**。
- **简单的字符相加:** $Hash(S) = (\sum_i ASCII(c_i)) \pmod{TableSize}$ 。
 - **问题:** 键短时分布不均匀; 不同字符组合 (如 "abc", "bca", "cab") 可能哈希到相同的值。
- **基数乘法 (Horner's Rule):** 将字符串 $c_1 c_2 \dots c_n$ 视为一个 **256 进制数**，使用霍纳法则进行计算，有效结合了字符的位置和值。

$$r = 0$$

for $i = 1$ to n do

✨ 冲突解决 (Collision Resolution)

当两个不同的键哈希到同一个索引时，需要冲突解决策略。

1. 分离链接法 (Separate Chaining)

- **方法:** 哈希表的每个槽位 (cell) 存储一个指向**链表** (或其他数据结构, 如 BST) 的指针。
 - **冲突:** 将新项插入到对应槽位的链表中。
 - **查找:** 计算哈希值, 然后在对应的链表上进行查找。
- **性能:** 平均时间复杂度为 $O(1) + O(\lambda)$, 其中 λ 是**负载系数 (Load Factor)**。
 - **负载系数** $\lambda = N / TableSize$, 其中 N 是存储的项数。
 - λ 也是链表的**平均长度**。

2. 开放寻址法 (Open Addressing / Probing)

- **方法:** 不使用链接, 所有键都存储在哈希表本身中。
 - **冲突:** 查找另一个空槽位存储该项, 查找空槽位的序列称为**探测序列 (probe sequence)**。
- **探测序列** $h_i(X) = (Hash(X) + p(X, i)) \pmod{TableSize}$ 。 $p(X, i)$ 是冲突解决函数。
- **主要寻址方案:**

方案	探测函数 $p(X, i)$	描述	缺点
线性探测 (Linear Probing)	$p(X, i) = i$	依次检查 $h(K), h(K) + 1, h(K) + 2, \dots \pmod{M}$ 。	一级聚集 (Primary Clustering): 随着聚集成块, 后续的冲突会倾向于落入这些块中, 使聚集块不断增大。
二次探测 (Quadratic Probing)	$p(K, i) = i^2$	检查 $h(K), h(K) + 1^2, h(K) + 2^2, \dots \pmod{M}$ 。	消除了一级聚集, 但有 二级聚集 (Secondary Clustering): 散列到相同 主位置 (home position) 的键会遵循相同的探测序列。
双重哈希 (Double Hashing)	$p(K, i) = i \cdot Hash_2(K)$	检查 $h_1(X), h_1(X) + h_2(X), h_1(X) + 2 \cdot h_2(X), \dots \pmod{M}$ 。	提供了更好的随机性, 能够有效减少聚集。 要求 $Hash_2(X)$ 既不能为 0, 也不能是 M 的因子。

3. 再哈希 (Rehashing)

当哈希表过满 (例如 $\lambda \approx 1$) 或发生大量删除 (使用**惰性删除**时) 导致性能下降时, 需要重建表格。

- **方法:**
 1. 建立一个**大约两倍大**的新哈希表。
 2. 遍历旧表中的非删除项。
 3. **重新计算**每个键在新表中的哈希值并插入。

- **触发时机:** 通常采用**中间策略 (middle-of-the-road strategy)**，即当负载系数达到某一特定值时触发。
 - **性能:** $O(N)$ 的时间复杂度，但发生频率很低。
-

开放寻址分析 (Analysis of Open Addressing)

- **经验法则:** 设计哈希系统时，应确保哈希表**永远不要超过半满**。
- **空表 (Almost Empty):** 插入、删除和查找操作通常只需要**一次**记录访问。
- **趋满 (Getting Full):** 预期成本 T 是负载系数 λ 的函数 $f(\lambda)$ 。
 - **随机探测 (Random Probing):** 预期的探针次数约为 $1/(1 - \lambda)$ 。
 - **线性探测 (Linear Probing):** 插入的平均成本约为 $\frac{1}{2} \left(1 + \frac{1}{(1-\lambda)^2} \right)$ ，删除的平均成本约为 $\frac{1}{2} \left(1 + \frac{1}{1-\lambda} \right)$ 。

希望这个总结对您有所帮助！您想让我进一步解释**线性探测中一级聚集**的图例 (如 PPT 47 页所示) 吗？